

24기 김려원 6주차 과제

Attention Is All You Need (Transformer) — 논문 스터디 정리

Vaswani et al., 2017 (NIPS 2017) | Google Brain / Google Research
arXiv:1706.03762

1. 한 줄 요약

RNN도, CNN도 없이 오직 Attention만으로 구성된 시퀀스 변환(sequence transduction) 모델 **Transformer**를 제안. 병렬화가 가능해 학습 속도가 훨씬 빠르면서도, 기계번역(WMT 2014 En-De, En-Fr)에서 당시 최고 성능(SOTA)을 달성.

2. 배경 및 문제의식

기존 시퀀스 모델(RNN, LSTM, GRU)은 **순차적으로 계산**해야 하므로:

병렬화가 불가능 → 학습이 느림

시퀀스가 길어질수록 메모리 제약으로 배치 처리도 어려움

Attention 메커니즘은 이미 여러 모델에 쓰이고 있었지만, 대부분 **RNN과 함께** 보조적으로 사용됨

핵심 아이디어: RNN/CNN을 완전히 제거하고, 입력과 출력 간의 전역 의존성을 **Attention만으로** 학습하자

왜 Self-Attention인가?

레이어 종류	레이어당 복잡도	순차 연산 수	최대 경로 길이
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent(RNN)	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$

Self-Attention은 **모든 위치 쌍을 상수 횟수의 연산으로 직접 연결** → 장거리 의존성 학습에 유리

시퀀스 길이 n 이 표현 차원 d 보다 작은 경우(문장 표현에서 일반적) self-attention이 RNN보다 계산상 더 빠름

3. 모델 구조 (Transformer Architecture)

3.1 전체 구조

Encoder-Decoder 구조 유지 (기존 seq2seq와 동일한 큰 틀)

Encoder: 동일한 레이어 **N=6개** 스택

서브레이어 1: Multi-Head Self-Attention

서브레이어 2: Position-wise Feed-Forward Network

각 서브레이어에 **residual connection + LayerNorm**: $\text{LayerNorm}(x + \text{Sublayer}(x))$

Decoder: 역시 N=6개 스택, Encoder와 유사하지만 서브레이어가 3개

Masked Multi-Head Self-Attention (미래 위치를 못 보게 마스킹 → auto-regressive 보장)

Encoder-Decoder Attention (Query는 디코더, Key/Value는 인코더 출력)

Feed-Forward Network

모든 서브레이어와 임베딩의 출력 차원: **d_model = 512**

3.2 Attention 메커니즘 (논문의 핵심)

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Query, Key, Value 세 벡터 집합 간의 연산

$Q \cdot K$ 의 내적으로 "유사도(호환성)"를 구하고, softmax로 가중치화 후 V에 곱해 가중합

$\sqrt{d_k}$ 로 스케일링하는 이유: d_k 가 커지면 내적 값의 분산이 커져 softmax가 극단적인 영역 (gradient가 매우 작은 영역)으로 밀려남 → 이를 방지하기 위해 스케일링

Multi-Head Attention

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \\ \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

하나의 큰 attention 대신, Q/K/V를 **h=8개의 서로 다른 학습된 선형 투영**으로 나눠 병렬로 attention 수행 후 concat

각 head의 차원: $d_k = d_v = d_{\text{model}}/h = 64$

의미: 하나의 attention만 쓰면 평균화 효과로 정보가 희석됨 → 여러 head가 서로 다른 표현 부분공간(subspace)에서 각기 다른 패턴(문법적/의미적 관계 등)을 학습 가능

Attention이 쓰이는 3가지 위치

Encoder-Decoder Attention: Query=디코더, Key/Value=인코더 출력 → 디코더가 입력 전체를 참조

Encoder Self-Attention: 인코더 내부에서 모든 위치가 서로를 참조

Decoder Self-Attention: 디코더 내부, **미래 위치는 마스킹**(auto-regressive 속성 유지)

3.3 Position-wise Feed-Forward Network

$$FFN(x) = \max(0, xW1 + b1)W2 + b2$$

각 위치에 독립적/동일하게 적용되는 2층 선형변환 + ReLU

내부 차원 d_ff = 2048 (입출력은 d_model=512)

3.4 Positional Encoding

Transformer는 RNN/CNN이 없어 **순서 정보를 알 수 없음** → 입력 임베딩에 위치 정보를 더해줌

sin/cos 함수 사용:

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$$

학습된 위치 임베딩과 성능이 거의 동일했지만, **sin/cos을 선택한 이유:** 학습 때보다 더 긴 시퀀스 길이에 확장(extrapolate) 가능할 것으로 기대했기 때문

4. 학습 설정

데이터: WMT 2014 En-De(약 450만 문장쌍), En-Fr(3600만 문장쌍), Byte-Pair/Word-Piece 인코딩

Optimizer: Adam ($\beta_1=0.9$, $\beta_2=0.98$, $\epsilon=1e-9$)

Learning rate warmup: 처음 4000 스텝은 선형 증가, 이후 step number의 역제곱근에 비례해 감소

정규화: Residual Dropout(0.1), Label Smoothing(0.1)

하드웨어: P100 GPU 8개, base 모델은 12시간, big 모델은 3.5일 학습

5. 실험 결과

모델	En-De BLEU	En-Fr BLEU	학습 비용(FLOPs)
기존 SOTA 양상블 모델들	~26.3	~41.3	매우 큼
Transformer (base)	27.3	38.1	3.3×10^{18} (훨씬 적음)
Transformer (big)	28.4	41.8	2.3×10^{19}

En-De에서 기존 최고 양상블 모델보다 **+2.0 BLEU 이상 향상**, 훨씬 적은 학습 비용으로 달성

Ablation(Table 3) 주요 발견:

Head 수: 너무 적어도(1개), 너무 많아도 성능 저하 (8개 근처가 적절)

d_k (key 차원)를 줄이면 성능 하락 → 단순 내적보다 더 정교한 호환성 함수가 유리할 수 있음

모델이 클수록, dropout을 적절히 쓸수록 성능 향상

sin/cos positional encoding ↔ 학습된 위치 임베딩: 성능 거의 동일

일반화 실험: 영어 구문 분석(Constituency Parsing)에도 적용 → 소량 데이터(WSJ 4만 문장)에서도 기존 RNN 기반 seq2seq보다 우수한 F1 달성 (task-specific 튜닝 없이도)

6. 의의 및 이후 영향

RNN/CNN 없이 attention만으로 SOTA 달성을 처음 증명한 논문

완전 병렬화 가능 → 학습 속도·효율 대폭 개선

Self-attention은 모든 위치를 $O(1)$ 경로로 연결 → 장거리 의존성 학습에 유리

Multi-head 구조가 **해석 가능성(interpretability)** 도 제공 (attention 시각화에서 head 별로 구문/의미 구조를 포착하는 패턴 관찰)

이후 BERT, GPT 시리즈 등 **현대 LLM의 근간 아키텍처**가 됨